

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Алгоритм Ахо-Корасик

Студент гр. 4343

Едыгеев Р. Б.

Преподаватель

Калишенко Е. Л.

Санкт-Петербург

2026

Цель работы.

Необходимо реализовать программу, находящую все вхождения всех образцов P_i в тексте T .

Работа выполнена в соответствии с вариантом 1.

Задание 1.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T , $1 \leq |T| \leq 100000$). Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Выход:

Все вхождения образцов из PP в TT . Каждое вхождение образца в текст представить в виде двух чисел – i p , где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1). Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Задание 2.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*. В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблон образцу P необходимо найти все вхождения P в текст T . Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababсах$. Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е.

шаблоны вида ??? недопустимы. Все строки содержат символы из алфавита {A, C, G, T, N}

Вход:

Текст ($T, 1 \leq |T| \leq 100000T$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Задание 3.

Модифицировать программу для задания 2 таким образом, чтобы можно было устанавливать символ, запрещенный для размещения вместо символа джокера.

Основные теоретические положения.

Алгоритм Ахо-Корасик решает задачу множественного поиска: найти все вхождения набора образцов в текст за линейное время. Он объединяет идеи префиксного дерева (бора) и конечного автомата.

Сначала все образцы добавляются в бор — дерево, где каждый путь от корня соответствует некоторому префиксу образца, а терминальные вершины помечаются номерами образцов.

Затем строится автомат: для каждой вершины вычисляется суффиксная (failure) ссылка, ведущая в вершину, соответствующую самому длинному собственному суффиксу текущего пути, который также является префиксом какого-либо образца. Для корня и его детей failure-ссылка указывает на корень. Остальные failure-ссылки находятся обходом в ширину: для вершины v , полученной из родителя u по символу s , failure-ссылка равна переходу из failure-ссылки u по тому же символу s . Если такого перехода нет, идём по failure-ссылкам, пока не найдём.

Дополнительно строятся терминальные ссылки (output links), которые ведут от вершины к ближайшей терминальной вершине, достижимой по failure-ссылкам. Это позволяет быстро собрать все образцы, заканчивающиеся в текущей позиции, без повторных подъёмов.

Поиск по тексту выполняется за один проход: поддерживается текущее состояние автомата, для каждого символа текста выполняются переходы по рёбрам или failure-ссылкам при отсутствии ребра. В каждой позиции по терминальным ссылкам из текущего состояния собираются все найденные образцы, вычисляется их начальная позиция и сохраняется в результат.

Выполнение работы.

Было реализовано три программы на языке Python – simple.py, joker.py, mod.py (см. Приложения А) для выполнения соответственно трех заданий. Также были созданы копии программ с логированием, хранящиеся в папке log.

- **simple.py.**

Описание структуры.

Класс AhoCorasick - основной класс, реализующий алгоритм. Содержит следующие структуры данных:

1. trie - список словарей, где каждый словарь хранит переходы {символ: индекс узла}. Индекс в списке — номер узла.
2. fail - список failure-ссылок для каждого узла (куда идти при несовпадении).
3. terminal_link - список терминальных ссылок (ведут к следующему терминальному узлу).
4. Terminal - булев список, отмечающий терминальные узлы.
5. Output - список списков номеров образцов, заканчивающихся в каждом узле.

Методы класса.

`add_pattern(pattern, index)` - добавляет образец в бор. Проходит по символам, создаёт новые узлы при необходимости, последний узел помечает терминальным и сохраняет номер образца.

`build_links()` - строит failure-ссылки обходом в ширину (BFS) и терминальные ссылки. Для каждого узла вычисляет fail по ссылке родителя, а также определяет `terminal_link`: если fail-узел терминальный — ссылается на него, иначе повторяет его `terminal_link`.

`get_all_patterns(node)` - собирает все образцы в узле и по цепочке `terminal_link`. Возвращает список номеров образцов.

`search(text, pattern_lengths)` - проходит по тексту, поддерживая текущее состояние. При несовпадении идёт по fail-ссылкам. При совпадении переходит по ребру. В каждой позиции вызывает `get_all_patterns` и сохраняет найденные вхождения с вычисленной начальной позицией.

`show()` - отладочный метод, записывает в файл структуру бора.

Пайплайн.

На вход программе подаются данные в формате:

T: text

N: number of patterns

$P_i, I = 1, \dots, N$: patterns

Из имеющихся данных создается бор из паттернов P , затем строится автомат, для каждого узла вычисляются суффиксные и терминальные ссылки. После происходит поиск по тексту T . Результаты сортируются и выводятся на экран.

Оценка сложности.

Построение бора в худшем случае (когда все паттерны начинаются с разных символов, т.е. у корня ровно N детей) имеет сложность $O(\sum_{i=1}^N \text{len}(P_i))$, ведь количество вершин бора не превышает $m+1$ узел. При построении суффиксных и терминальных ссылок используется BFS $O(\sum_{i=1}^N \text{len}(P_i))$, в

каждом узле необходимо просмотреть fail-ссылку родителя и проверить, есть ли символ в текущую вершину и, при необходимости, спуститься по ссылке. Тогда сложность зависит от количества возможных спусков по ссылкам. Но при спуске по fail-ссылке уменьшается глубина спуска узла, т.е. в следующий раз при спуске в данный узел глубина уже будет меньше. Значит количество спусков будет линейно относительно количества узлов, значит сложность равна $O(\sum_{i=1}^N \text{len}(P_i))$.

Проход по тексту занимает $O(\text{len}(T))$ с учётом амортизации, так как переходы по fail-ссылкам при несовпадении суммарно дают линейное время. Для каждой позиции сбор всех найденных образцов выполняется за $O(\text{Кол-во совпадений})$ с помощью предвычисленных терминальных ссылок (terminal_link), которые ведут от вершины к следующей терминальной вершине. Таким образом, общая сложность поиска составляет $O(\text{len}(T) + \text{кол-во совпадений})$.

Суммарная сложность - $O(\sum_{i=1}^N \text{len}(P_i) + \text{len}(T) + \text{кол-во совпадений})$

Сложность по памяти состоит из хранения бора $O(\sum_{i=1}^N \text{len}(P_i) + \text{len}(T))$, ведь каждый символ каждого образца требует $O(1)$ памяти для хранения узла и рёбер, а дополнительные служебные структуры (fail-ссылки, терминальные ссылки, флаги) также линейны относительно количества узлов.

- **joker.py.**

Описание структуры.

Класс AhoCorasick - основной класс, реализующий алгоритм. Содержит следующие структуры данных:

6. trie - список словарей, где каждый словарь хранит переходы {символ: индекс узла}. Индекс в списке — номер узла.

7. fail - список failure-ссылок для каждого узла (куда идти при несовпадении).

8. `terminal_link` - список терминальных ссылок (ведут к следующему терминальному узлу).
9. `Terminal` - булев список, отмечающий терминальные узлы.
10. `Output` - список списков номеров образцов, заканчивающихся в каждом узле.

Методы класса.

`add_pattern(pattern, index)` - добавляет образец в бор. Проходит по символам, создаёт новые узлы при необходимости, последний узел помечает терминальным и сохраняет номер образца.

`build_links()` - строит failure-ссылки обходом в ширину (BFS) и терминальные ссылки. Для каждого узла вычисляет fail по ссылке родителя, а также определяет `terminal_link`: если fail-узел терминальный — ссылается на него, иначе повторяет его `terminal_link`.

`get_all_patterns(node)` - собирает все образцы в узле и по цепочке `terminal_link`. Возвращает список номеров образцов.

`show()` - отладочный метод, записывает в файл структуру бора.

Функция `find_pattern_with_wildcards(text, pattern, wildcard_char)` – поиск шаблона с джокерами, выделение непрерывных участков без джокеров (subpatterns) с запоминанием их стартовых позиций, проход по тексту с использованием автомата, подсчёт совпадений в массиве `C` и возврат ответа.

Пайплайн.

На вход программе подаются данные в формате:

T: text

P: pattern with wild cards

W: character of wild card

Происходит разбиение шаблона на подстроки без джокеров, из которых создается бор, затем строится автомат, для каждого узла вычисляются суффиксные и терминальные ссылки. Происходит проход по тексту с сохранением в массиве индекса начала паттерна (количество встретившихся в

тексте безмасочных подстрок шаблона, который начинается в тексте на данном индексе). Из массива C строится ответ.

Оценка сложности.

Алгоритм не имеет существенных различий в сложности с стандартным алгоритмом Ахо-Корасика – после разбиения шаблона на подстроки также создается бор, также строится автомат. Последующая пробежка по массиву C занимает $O(\text{len}(T))$, что не оказывает существенной разницы на итоговую сложность.

Суммарная сложность - $O(\text{len}(P) + \text{len}(T) + \text{кол-во совпадений})$.

Пространственная сложность равна хранению бора + доп. служебные структуры + массив C длиной не более длины текста – $O(\text{len}(T))$. Итоговая пространственная сложность - $O(\text{len}(P) + \text{len}(T))$.

• mod.py.

Описание структуры.

Класс AhoCorasick - основной класс, реализующий алгоритм. Содержит следующие структуры данных:

11. `trie` - список словарей, где каждый словарь хранит переходы {символ: индекс узла}. Индекс в списке — номер узла.
12. `fail` - список failure-ссылок для каждого узла (куда идти при несовпадении).
13. `terminal_link` - список терминальных ссылок (ведут к следующему терминальному узлу).
14. `Terminal` - булев список, отмечающий терминальные узлы.
15. `Output` - список списков номеров образцов, заканчивающихся в каждом узле.

Методы класса.

`add_pattern(pattern, index)` - добавляет образец в бор. Проходит по символам, создаёт новые узлы при необходимости, последний узел помечает терминальным и сохраняет номер образца.

`build_automaton()` - строит failure-ссылки обходом в ширину (BFS) и терминальные ссылки. Для каждого узла вычисляет fail по ссылке родителя, а также определяет `terminal_link`: если fail-узел терминальный — ссылается на него, иначе повторяет его `terminal_link`.

`get_all_patterns(node)` - собирает все образцы в узле и по цепочке `terminal_link`. Возвращает список номеров образцов.

`show()` - отладочный метод, записывает в файл структуру бора.

Функция `find_pattern_with_wildcards_excluded(text, pattern, wildcard_char)` – поиск шаблона с джокерами, исключая запрещенный символ, выделение непрерывных участков без джокеров (`subpatterns`) с запоминанием их стартовых позиций, проход по тексту с использованием автомата, подсчёт совпадений в массиве `C` и возврат ответа.

Пайплайн.

На вход программе подаются данные в формате:

T: text

P: pattern with wild cards

W: character of wild card

F: forbidden character for wild card

Происходит разбиение шаблона на подстроки без джокеров, из которых создается бор, затем строится автомат, для каждого узла вычисляются суффиксные и терминальные ссылки. Происходит проход по тексту с сохранением в массиве индекса начала паттерна (количество встретившихся в тексте безмасочных подстрок шаблона, который начинается в тексте на данном индексе), при том на каждом джокере происходит проверка, что символ из текста не соответствует запрещенному символу F. Из массива `C` строится ответ.

Оценка сложности.

Алгоритм не имеет существенных различий в сложности с предыдущим алгоритмом – единственное, на каждом джокере происходит проверка на запрещенный символ. Но т.к. каждый джокер проверяется не более одного раза на каждое найденное совпадение, итоговая сложность остается линейной.

Суммарная сложность - $O(\text{len}(P) + \text{len}(T) + (\text{кол-во совпадений}) * (\text{кол-во джокеров в паттерне}))$.

Пространственная сложность равна хранению бора + доп. служебные структуры + массив S длиной не более длины текста – $O(\text{len}(T))$. Итоговая пространственная сложность - $O(\text{len}(P) + \text{len}(T))$.

Тестирование.

Для программ было проведено тестирование, затрагивающее крайние случаи. Для начала приведено логирование программ (Рис. 1 – 5):

```

AHO-CORASICK DEBUG LOG
=====
✓ INPUT DATA:
  - Text: 'd098c789b456a123'
  - Patterns count: 5
    ... 1: 'a123'
    ... 2: 'b456'
    ... 3: 'c789'
    ... 4: 'd098'
    ... 5: 'e'

=====
      ADDING PATTERNS
=====
✓ [ADD] pattern 1: 'a123'
  .....create node 1 via 'a' from 0
  .....create node 2 via '1' from 1
  .....create node 3 via '2' from 2
  .....create node 4 via '3' from 3
  .....node 4 is TERMINAL for pattern 1
✓ [ADD] pattern 2: 'b456'
  .....create node 5 via 'b' from 0
  .....create node 6 via '4' from 5
  .....create node 7 via '5' from 6
  .....create node 8 via '6' from 7
  .....node 8 is TERMINAL for pattern 2
✓ [ADD] pattern 3: 'c789'
  .....create node 9 via 'c' from 0
  .....create node 10 via '7' from 9
  .....create node 11 via '8' from 10
  .....create node 12 via '9' from 11
  .....node 12 is TERMINAL for pattern 3
✓ [ADD] pattern 4: 'd098'
  .....create node 13 via 'd' from 0
  .....create node 14 via '0' from 13
  .....create node 15 via '9' from 14

```

Рисунок 1. Логирование simple.py (общий шаблон)

```

=====
BUILDING FAILURE & TERMINAL LINKS
=====

[INIT] root children:
.....node 1 (via 'a'): fail=0, term_link=0
.....node 5 (via 'b'): fail=0, term_link=0
.....node 9 (via 'c'): fail=0, term_link=0
.....node 13 (via 'd'): fail=0, term_link=0
.....node 17 (via 'e'): fail=0, term_link=0

[STEP 1] process node 1
.....fail[1] = 0
.....term_link[1] = 0
.....edge '1' --> 2
.....start f = 0
.....'1' not found, fail = 0
.....fail[2] = 0

[STEP 2] process node 5
.....fail[5] = 0
.....term_link[5] = 0
.....edge '4' --> 6
.....start f = 0
.....'4' not found, fail = 0
.....fail[6] = 0

[STEP 3] process node 9
.....fail[9] = 0
.....term_link[9] = 0
.....edge '7' --> 10
.....start f = 0
.....'7' not found, fail = 0
.....fail[10] = 0

```

Рисунок 2. Логирование simple.py (общий шаблон)

NODE DETAILS:						
Node	Terminal	Patterns	Transitions	fail	term_link	
0	•	[]	a→1, b→5, c→9, d→13...	0	0	
1	•	[]	1→2	0	0	
2	•	[]	2→3	0	0	
3	•	[]	3→4	0	0	
4	✓	[1]		0	0	
5	•	[]	4→6	0	0	
6	•	[]	5→7	0	0	
7	•	[]	6→8	0	0	
8	✓	[2]		0	0	
9	•	[]	7→10	0	0	
10	•	[]	8→11	0	0	
11	•	[]	9→12	0	0	
12	✓	[3]		0	0	
13	•	[]	0→14	0	0	
14	•	[]	9→15	0	0	
15	•	[]	8→16	0	0	
16	✓	[4]		0	0	
17	✓	[5]		0	0	

TREE VIEW:	
Node0	
└─ 'a' → 1	
└─ Node1	
└─ '1' → 2	
└─ Node2	
└─ '2' → 3	
└─ Node3	
└─ '3' → 4	
└─ Node4 [*] [1]	

Рисунок 3. Логирование simple.py (общий шаблон)

```

=====
SEARCH IN TEXT
=====
Text: 'd098c789b456a123'

[pos 1] char='d' state=0
|..... edge 'd' -> 13

[pos 2] char='0' state=13
|..... edge '0' -> 14

[pos 3] char='9' state=14
|..... edge '9' -> 15

[pos 4] char='8' state=15
|..... edge '8' -> 16
|..... FOUND patterns: [4]
|.....|....pattern 4 at position 1

[pos 5] char='c' state=16
|..... follow fail -> 0
|..... edge 'c' -> 9

[pos 6] char='7' state=9
|..... edge '7' -> 10

[pos 7] char='8' state=10
|..... edge '8' -> 11

[pos 8] char='9' state=11
|..... edge '9' -> 12
|..... FOUND patterns: [3]
|.....|....pattern 3 at position 5

[pos 9] char='b' state=12
|..... follow fail -> 0
|..... edge 'b' -> 5

```

Рисунок 4. Логирование simple.py, поиск

```

✓ [pos 7] char='w' state=1
  ..... edge 'w' -> 2
✓ ..... FOUND subpatterns: [(0, 0, 2)]
  ..... increment C[5] = 1

✓ [pos 8] char='r' state=2
  ..... follow fail -> 0
  ..... edge 'r' -> 1

✓ [pos 9] char='w' state=1
  ..... edge 'w' -> 2
✓ ..... FOUND subpatterns: [(0, 0, 2)]
  ..... increment C[7] = 1

✓ [pos 10] char='g' state=2
  ..... follow fail -> 0
  ..... no edge, stay at 0

✓ [pos 11] char='v' state=0
  ..... no edge, stay at 0

✓ [pos 12] char='w' state=0
  ..... no edge, stay at 0

=====
STEP 4: COLLECT RESULTS
=====
✓ C array (first 10 positions): [0, 0, 1, 0, 0, 1, 0, 1, 0, 0]

  position 3: C[2] = 1 == 1 -> MATCH
  position 6: C[5] = 1 == 1 -> MATCH
  position 8: C[7] = 1 == 1 -> MATCH

Total matches: 3
=====

```

Рисунок 5. Логирование joker.py, поиск

Тестирование программы simple.py (табл. 1):

Таблица 1. Тестирование simple.py

Вход	Выход	Комментарий
abc 3 ab, bc, abc	1 1 1 3 2 2	Простой тест работоспособности программы
aaa 2 aa aa	1 1 1 2 2 1 2 2	Работа с перекрытиями, когда паттерны пересекаются в тексте
defede 2 Xyz abc	[]	Проверка на отсутствие совпадений
[]	[]	Проверка на отсутствие ввода

Тестирование программы joker.py (табл. 2):

Таблица 2. Тестирование joker.py

Вход	Выход	Комментарий
abcdefabc abc ?	1, 7	Работа программы при шаблоне без джокеров
abcxabcsyabcz ab? ?	1, 5, 9	Простой тест работоспособности программы
xyzabcxyz ?abc? ?	3	Проверка на крайние положения джокеров
[]	[]	Проверка на отсутствие ввода
Abcbacba ??? ?	[]	Паттерн полностью из джокеров

Тестирование программы mod.py (табл. 3):

Таблица 3. Тестирование mod.py

Вход	Выход	Комментарий
abcdefabc abc ? c	1, 7	Работа программы при шаблоне без джокеров
abcxabbyabcz ab? ? c	5	Простой тест работоспособности программы
xyzabcxyz ?abc? ? l	3	Проверка на крайние положения джокеров
[]	[]	Проверка на отсутствие ввода

Abcbacba ??? ? a	[]	Паттерн полностью из джокеров
---------------------------	-----	----------------------------------

Вывод.

В ходе выполнения лабораторной работы был изучен алгоритм Ахо-Корасика для нахождения всех вхождений всех подстрок в текст, а также решение данной задачи с усложнением – джокерами и запрещенным символами. Были разработаны программы на Python, выполнен анализ сложности алгоритма по времени и памяти, проведено тестирование программ, осуществлено логирование процесса выполнения программы.

Был получен опыт построения алгоритма для решения задачи, а также опыт в сфере оценки и анализа проведенной работы.

ПРИЛОЖЕНИЕ А КОД ПРОГРАММЫ

Файл simple.py:

```
from collections import deque

log_file = "log_1.txt"

class AhoCorasick:
    def __init__(self):
        self.trie = [{}]
        self.fail = [0]
        self.terminal_link = [0]
        self.terminal = [False]
        self.output = [[]]

    def add_pattern(self, pattern, index):
        node = 0
        for char in pattern:
            if char not in self.trie[node]:
                self.trie[node][char] = len(self.trie)
                self.trie.append({})
                self.fail.append(0)
                self.terminal_link.append(0)
                self.terminal.append(False)
                self.output.append([])
            node = self.trie[node][char]
        self.terminal[node] = True
        self.output[node].append(index)

    def build_links(self):
        q = deque()

        for char, next_node in self.trie[0].items():
            self.fail[next_node] = 0
            q.append(next_node)

        self.terminal_link[next_node] = 0
        while q:
            current = q.popleft()
            if self.terminal[self.fail[current]]:
                self.terminal_link[current] = self.fail[current]
            else:
                self.terminal_link[current] = self.terminal_link[self.fail[current]]

            for char, next_node in self.trie[current].items():
                q.append(next_node)
                f = self.fail[current]
                while f != 0 and char not in self.trie[f]:
                    f = self.fail[f]

                if char in self.trie[f]:
                    self.fail[next_node] = self.trie[f][char]
                else:
                    self.fail[next_node] = 0
```

```

def get_all_patterns(self, node):
    patterns = []
    if self.terminal[node]:
        patterns.extend(self.output[node])
    current = self.terminal_link[node]
    while current != 0:
        patterns.extend(self.output[current])
        current = self.terminal_link[current]
    return patterns

def search(self, text, pattern_lengths):
    results = []
    node = 0

    for i, char in enumerate(text):
        while node != 0 and char not in self.trie[node]:
            node = self.fail[node]
        if char in self.trie[node]:
            node = self.trie[node][char]
        else:
            node = 0
        for pattern_index in self.get_all_patterns(node):
            start_pos = i - pattern_lengths[pattern_index] + 2
            results.append((start_pos, pattern_index))
    return results

def show(self):
    f = open(log_file, "w")
    f.write(30 * "=" + "BOHR" + 30 * "=" + "\n")
    node = 0
    level = 0
    def dfs(node, level):
        leaf = self.trie[node]
        f.writelines("\t\t" * level + f"ID({node}) {leaf}" + f" |
isTerm - {self.terminal[node]}" + "\n" + level * "\t\t|" + "\n")
        level += 1
        for child in self.trie[node].values():
            dfs(child, level)
    dfs(node, level)
    f.close()

def main():
    text = input().strip()
    n = int(input().strip())
    patterns = []
    for _ in range(n):
        patterns.append(input().strip())

    ac = AhoCorasick()
    for i, pattern in enumerate(patterns, 1):
        ac.add_pattern(pattern, i)

    ac.build_links()
    ac.show()

```

```

        pattern_lengths = {i: len(pattern) for i, pattern in
enumerate(patterns, 1)}
        results = ac.search(text, pattern_lengths)

        results.sort()
        for pos, idx in results:
            print(pos, idx)

if __name__ == "__main__":
    main()

```

Файл joker.py:

```

from collections import deque

log_file = "log_2.txt"

class AhoCorasick:
    def __init__(self):
        self.trie = [{}]
        self.fail = [0]
        self.output_link = [0]
        self.terminal = [False]
        self.pattern_info = [[]]

    def add_pattern(self, pattern, pattern_index, start_pos_in_pattern,
length):
        node = 0
        for char in pattern:
            if char not in self.trie[node]:
                self.trie[node][char] = len(self.trie)
                self.trie.append({})
                self.fail.append(0)
                self.output_link.append(0)
                self.terminal.append(False)
                self.pattern_info.append([])
            node = self.trie[node][char]

        self.terminal[node] = True
        self.pattern_info[node].append((pattern_index,
start_pos_in_pattern, length))

    def build_automaton(self):
        q = deque()
        for char, next_node in self.trie[0].items():
            self.fail[next_node] = 0
            q.append(next_node)

        while q:
            current = q.popleft()
            if self.terminal[self.fail[current]]:
                self.output_link[current] = self.fail[current]
            else:
                self.output_link[current] =
self.output_link[self.fail[current]]

```

```

        for char, next_node in self.trie[current].items():
            q.append(next_node)
            f = self.fail[current]
            while f != 0 and char not in self.trie[f]:
                f = self.fail[f]

            if char in self.trie[f]:
                self.fail[next_node] = self.trie[f][char]
            else:
                self.fail[next_node] = 0

def get_all_patterns(self, node):
    patterns = []

    if self.terminal[node]:
        patterns.extend(self.pattern_info[node])

    current = self.output_link[node]
    while current != 0:
        patterns.extend(self.pattern_info[current])
        current = self.output_link[current]

    return patterns

def show(self):
    f = open(log_file, "w")
    f.write(30 * "=" + "BOHR" + 30 * "=" + "\n")
    node = 0
    level = 0
    def dfs(node, level):
        leaf = self.trie[node]
        f.writelines("\t\t" * level + f"ID({node}) {leaf}" + f" |
isTerm - {self.terminal[node]}" + "\n" + level * "\t\t|" + "\n")
        level += 1
        for child in self.trie[node].values():
            dfs(child, level)
    dfs(node, level)
    f.close()

def find_pattern_with_wildcards(text, pattern, wildcard_char):
    n = len(text)
    m = len(pattern)

    subpatterns = []
    i = 0
    while i < m:
        if pattern[i] != wildcard_char:
            start = i
            j = i
            while j < m and pattern[j] != wildcard_char:
                j += 1
            subpatterns.append((pattern[start:j], start))
            i = j
        else:
            i += 1

    k = len(subpatterns)

```

```

if k == 0 or m > n:
    return []

ac = AhoCorasick()
for idx, (subpattern, start_pos) in enumerate(subpatterns):
    ac.add_pattern(subpattern, idx, start_pos, len(subpattern))

ac.show()
ac.build_automaton()
C = [0] * n

node = 0

for pos, char in enumerate(text):
    while node != 0 and char not in ac.trie[node]:
        node = ac.fail[node]

    if char in ac.trie[node]:
        node = ac.trie[node][char]
    else:
        node = 0

    patterns = ac.get_all_patterns(node)

    for pattern_idx, start_pos_in_pattern, length in patterns:
        pattern_start_in_text = pos - length + 1 -
start_pos_in_pattern

        if 0 <= pattern_start_in_text < n:
            C[pattern_start_in_text] += 1

results = []
for i in range(n - m + 1):
    if C[i] == k:
        results.append(i + 1)

return results

def main():
    text = input().strip()
    pattern = input().strip()
    wildcard_char = input().strip()
    positions = find_pattern_with_wildcards(text, pattern,
wildcard_char)
    for pos in positions:
        print(pos)

if __name__ == "__main__":
    main()

```

Файл mod.py:

```

from collections import deque

log_file = "log_3.txt"

```

```

class AhoCorasick:
    def __init__(self):
        self.trie = [{}]
        self.fail = [0]
        self.output_link = [0]
        self.terminal = [False]
        self.pattern_info = [[]]

    def add_pattern(self, pattern, pattern_index, start_pos_in_pattern,
length):
        node = 0
        for char in pattern:
            if char not in self.trie[node]:
                self.trie[node][char] = len(self.trie)
                self.trie.append({})
                self.fail.append(0)
                self.output_link.append(0)
                self.terminal.append(False)
                self.pattern_info.append([])
            node = self.trie[node][char]

        self.terminal[node] = True
        self.pattern_info[node].append((pattern_index,
start_pos_in_pattern, length))

    def build_automaton(self):
        q = deque()

        for char, next_node in self.trie[0].items():
            self.fail[next_node] = 0
            q.append(next_node)

        while q:
            current = q.popleft()

            if self.terminal[self.fail[current]]:
                self.output_link[current] = self.fail[current]
            else:
                self.output_link[current] = self.output_link[self.fail[current]]

            for char, next_node in self.trie[current].items():
                q.append(next_node)

            f = self.fail[current]
            while f != 0 and char not in self.trie[f]:
                f = self.fail[f]

            if char in self.trie[f]:
                self.fail[next_node] = self.trie[f][char]
            else:
                self.fail[next_node] = 0

    def get_all_patterns(self, node):
        patterns = []
        if self.terminal[node]:

```



```

        patterns.extend(self.pattern_info[node])
    current = self.output_link[node]
    while current != 0:
        patterns.extend(self.pattern_info[current])
        current = self.output_link[current]
    return patterns

def show(self):
    f = open(log_file, "w")
    f.write(30 * "=" + "BOHR" + 30 * "=" + "\n")
    node = 0
    level = 0
    def dfs(node, level):
        leaf = self.trie[node]
        f.writelines("\t\t" * level + f"ID({node}) {leaf}" + f" |
isTerm - {self.terminal[node]}" + "\n" + level * "\t\t|" + "\n")
        level += 1
        for child in self.trie[node].values():
            dfs(child, level)
    dfs(node, level)
    f.close()

def find_pattern_with_wildcard_excluding(text, pattern, wildcard_char,
forbidden_char):
    n = len(text)
    m = len(pattern)
    subpatterns = []
    i = 0
    while i < m:
        if pattern[i] != wildcard_char:
            start = i
            j = i
            while j < m and pattern[j] != wildcard_char:
                j += 1
            subpatterns.append((pattern[start:j], start))
            i = j
        else:
            i += 1

    k = len(subpatterns)

    if k == 0 or m > n:
        return []

    ac = AhoCorasick()
    for idx, (subpattern, start_pos) in enumerate(subpatterns):
        ac.add_pattern(subpattern, idx, start_pos, len(subpattern))
    ac.show()

    ac.build_automaton()
    C = [0] * n
    node = 0

    for pos, char in enumerate(text):
        while node != 0 and char not in ac.trie[node]:
            node = ac.fail[node]
        if char in ac.trie[node]:

```

```

        node = ac.trie[node][char]
    else:
        node = 0
    patterns_found = ac.get_all_patterns(node)

    for pattern_idx, start_pos_in_pattern, length in patterns_found:
        pattern_start_in_text = pos - length + 1 -
start_pos_in_pattern
        if 0 <= pattern_start_in_text < n:
            valid = True
            j = 0
            current_pos_in_pattern = 0

            for subpattern, sub_start in subpatterns:
                while current_pos_in_pattern < sub_start:
                    text_pos = pattern_start_in_text +
current_pos_in_pattern
                    if text_pos < n and text[text_pos] ==
forbidden_char:
                        valid = False
                        break
                        current_pos_in_pattern += 1

                if not valid:
                    break

                current_pos_in_pattern += len(subpattern)

            while valid and current_pos_in_pattern < m:
                text_pos = pattern_start_in_text +
current_pos_in_pattern
                if text_pos < n and text[text_pos] == forbidden_char:
                    valid = False
                    break
                    current_pos_in_pattern += 1

            if valid:
                C[pattern_start_in_text] += 1

    results = []
    for i in range(n - m + 1):
        if C[i] == k:
            results.append(i + 1)

    return results

def main():
    text = input().strip()
    pattern = input().strip()
    wildcard_char = input().strip()
    forbidden_char = input().strip()

    positions = find_pattern_with_wildcard_excluding(
        text, pattern, wildcard_char, forbidden_char
    )

    for pos in positions:

```

```
        print(pos)

if __name__ == "__main__":
    main()
```